

MA. Actividad: Roomba

1. PROBLEMA Y PROPUESTA DE SOLUCIÓN

1.1 Descripción del Problema

Se requiere desarrollar un sistema de agentes autónomos capaces de limpiar eficientemente una habitación de dimensiones $M \times N$ espacios. Los agentes deben operar bajo las siguientes restricciones:

- Batería limitada (inicia en 100% y se consume 1% por acción)
- Necesidad de recarga periódica en estaciones fijas (5% por paso en estación)
- Presencia de obstáculos que limitan la movilidad
- Distribución aleatoria inicial de basura
- Tiempo máximo de ejecución (200 pasos)

1.2 Propuesta de Solución

Se propone implementar un sistema multi-agente basado en una **arquitectura de subsunción** con los siguientes componentes clave:

- **Mapeo interno progresivo:** Cada agente construye un mapa del entorno a medida que lo explora, clasificando cada celda como: desconocida (-1), libre (0), obstáculo (1), o visitada (2).
- **Exploración inteligente:** Uso de BFS (Búsqueda en Amplitud) para planificar rutas óptimas hacia objetivos (basura, estaciones de carga, fronteras inexploradas).
- **Gestión autónoma de energía:** Los agentes monitorean continuamente su batería y regresan proactivamente a cargar cuando alcanzan un umbral crítico (40%).
- **Coordinación implícita:** En el escenario multi-agente, se implementa detección de colisiones y sistema de espera para estaciones de carga compartidas.

2. DISEÑO DE LOS AGENTES

2.1 Tipos de agentes

Hay cuatro tipos de agentes inicializados en el sistema, pero solo los roombas (randomAgent) son “inteligentes” y poseen instrucciones complejas. Los agentes restantes: Obstáculos, estaciones de carga y basura son agentes fijos que sirven de algún propósito (obtención de energía, dificultad de movimiento y recolección) al agente principal que son los roombas o a la simulación.

2.2 Características del RandomAgent Roomba

Los roombas tienen como objetivo limpiar el máximo porcentaje posible de celdas sucias en el ambiente dentro del tiempo máximo establecido (200 pasos), mientras mantiene su nivel de energía por encima de 0% para evitar quedar inoperativo. Los agentes deben balancear tres sub-objetivos simultáneamente: (1) **Exploración eficiente** del ambiente para descubrir todas las áreas con basura, construyendo un mapa mental completo del espacio disponible; (2)

Recolección de basura mediante limpieza de cada celda sucia encontrada, priorizando basura visible sobre exploración de nuevas áreas; y (3) **Gestión autónoma de energía** monitoreando constantemente su batería y regresando proactivamente a la estación de carga cuando alcanza el

umbral crítico del 40%, para garantizar supervivencia operativa. En el escenario multi-agente, existe un objetivo implícito adicional de optimización colectiva, donde múltiples agentes deben coordinarse sin comunicación explícita para evitar redundancia excesiva en la exploración y maximizar la cobertura total del equipo.

Los agentes RandomAgent son altamente reactivos, respondiendo inmediatamente a estímulos del ambiente como la detección de basura adyacente (limpieza inmediata) o niveles críticos de batería (retorno urgente a estación de carga), adaptando su comportamiento según las percepciones actuales. Son también proactivos, ya que no solo reaccionan a situaciones inmediatas sino que planifican rutas hacia objetivos distantes usando BFS, anticipan necesidades futuras de energía (iniciando recarga antes de agotarse completamente), y mantienen un objetivo de largo plazo de maximizar la limpieza del ambiente. Poseen autonomía completa, tomando decisiones independientes sin intervención humana, gestionando su propia energía, navegación y estrategia de exploración. Implementan una arquitectura de subsunción con cinco niveles jerárquicos de comportamiento, donde comportamientos de mayor prioridad pueden subsumir (inhibir) los de menor prioridad. Son persistentes, operando continuamente hasta completar la tarea o alcanzar el límite de tiempo. En escenarios multi-agente, demuestran coordinación implícita mediante detección de colisiones y sistemas de espera en estaciones compartidas, aunque carecen de comunicación explícita entre agentes.

2.3 Métricas de desempeño

- **Porcentaje de celdas limpias:** $(\text{Celdas sin basura} / \text{Total de celdas}) \times 100$
- **Tiempo de limpieza total:** Número de pasos hasta completar la limpieza o alcanzar el límite
- **Movimientos totales:** Suma de todos los desplazamientos realizados por todos los agentes
- **Basura recolectada:** Número total de celdas sucias que fueron limpiadas
- **Energía promedio:** Nivel promedio de batería de todos los agentes
- **Eficiencia energética:** $\text{Basura recolectada} / \text{Energía total consumida}$

3. ARQUITECTURA DE SUBSUNCIÓN

<i>Gestión de energía</i>
<i>Limpieza reactiva</i>
<i>Exploración local</i>
<i>Exploración proactiva</i>

Evitar obstáculos

3.1 Descripción de niveles

Las capas van de mayor a menor, poniendo gestión de energía como la actividad más importante y evitar obstáculos como la última prioridad para el roomba:

Gestión de energía: Cuando la batería cae por debajo del 40%, este nivel subsume todos los comportamientos de exploración y limpieza, forzando al agente a calcular una ruta hacia la estación de carga más cercana. Una vez en la estación, el agente permanece cargando hasta alcanzar 100%.

Limpieza reactiva: Si el agente detecta basura en cualquier celda adyacente, este nivel subsume la exploración y dirige al agente inmediatamente hacia la basura para limpiarla. Esto asegura que la basura visible se recolecte de inmediato.

Exploración local: Cuando no hay basura inmediata pero existen celdas desconocidas (-1) en el vecindario, el agente se mueve hacia ellas para expandir su conocimiento del entorno.

Exploración proactiva: Cuando no hay estímulos locales interesantes, el agente busca en su mapa interno la celda inexplorada más cercana y planifica una ruta óptima usando BFS. Este es el comportamiento por defecto que permite una exploración sistemática del entorno.

Evitar obstáculos: Comportamiento básico de navegación. El agente verifica que las celdas no contengan obstáculos antes de moverse. Este nivel opera continuamente pero es fácilmente subsumido por niveles superiores.

4. CARACTERÍSTICAS DEL AMBIENTE

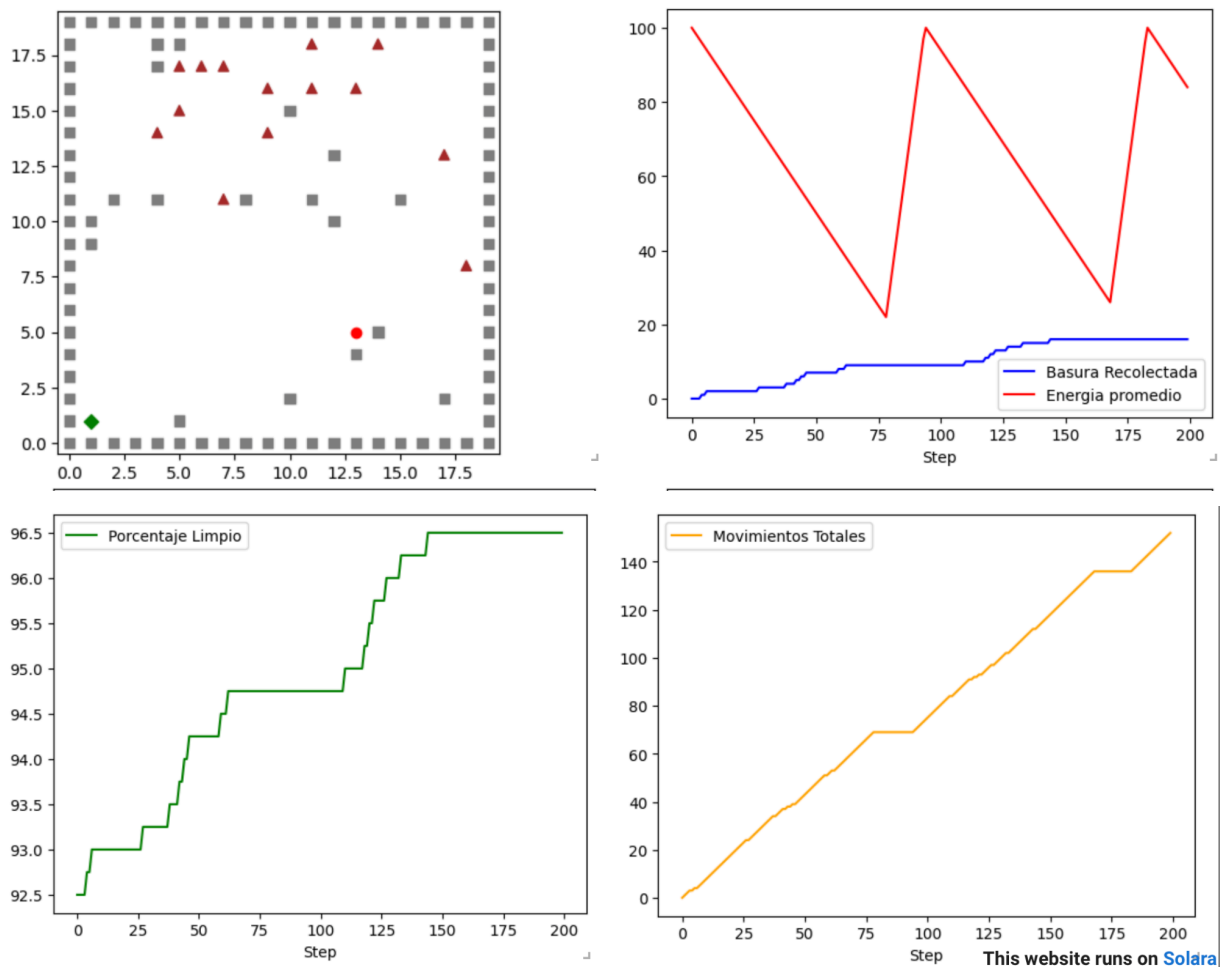
El ambiente está basado en una grilla ortogonal de dimensiones $M \times N$ con bordes que actúan como obstáculos infranqueables. Los agentes consideran 8 vecinos (vecindario de Moore) para percepción y movimiento. El ambiente es parcialmente observable, ya que cada agente solo tiene información de su vecindario inmediato y debe construir progresivamente un mapa interno del entorno a medida que lo explora, sin conocimiento global de la distribución de basura u obstáculos. Es un ambiente no episódico y secuencial, donde las decisiones en cada paso afectan directamente los pasos siguientes (la energía consumida, las celdas visitadas y la basura recolectada persisten). El ambiente es estático desde la perspectiva de cambios externos, ya que solo se modifica por las acciones de los agentes (limpieza de basura, consumo de batería, movimiento), no cambia espontáneamente durante la deliberación del agente. Es discreto tanto en espacio (celdas de grilla) como en tiempo (pasos de simulación), con acciones bien definidas y estados finitos. El ambiente no es determinista ya que debido a la aleatoriedad de las estaciones de carga y obstáculos, la simulación cambia cada vez que la reseteamos.

5. SIMULACIONES

Se realizaron dos repeticiones por simulación, en las cuales en el agente simple comenzamos con una grilla de 20x20, un porcentaje de obstáculos de 6% y un porcentaje de celdas sucias de 10% y en la segunda simulación aumentamos las cantidades a una grilla de 35x35, un porcentaje de obstáculos del 10% y 18% de celdas sucias. En las simulaciones de múltiples agentes comenzamos con una grilla de 20x20 y 10 agentes, y en la segunda simulación tenemos una grilla de 35x35 y 20 agentes. En todas las simulaciones la cantidad máxima de steps es de 200.

Agente individual:

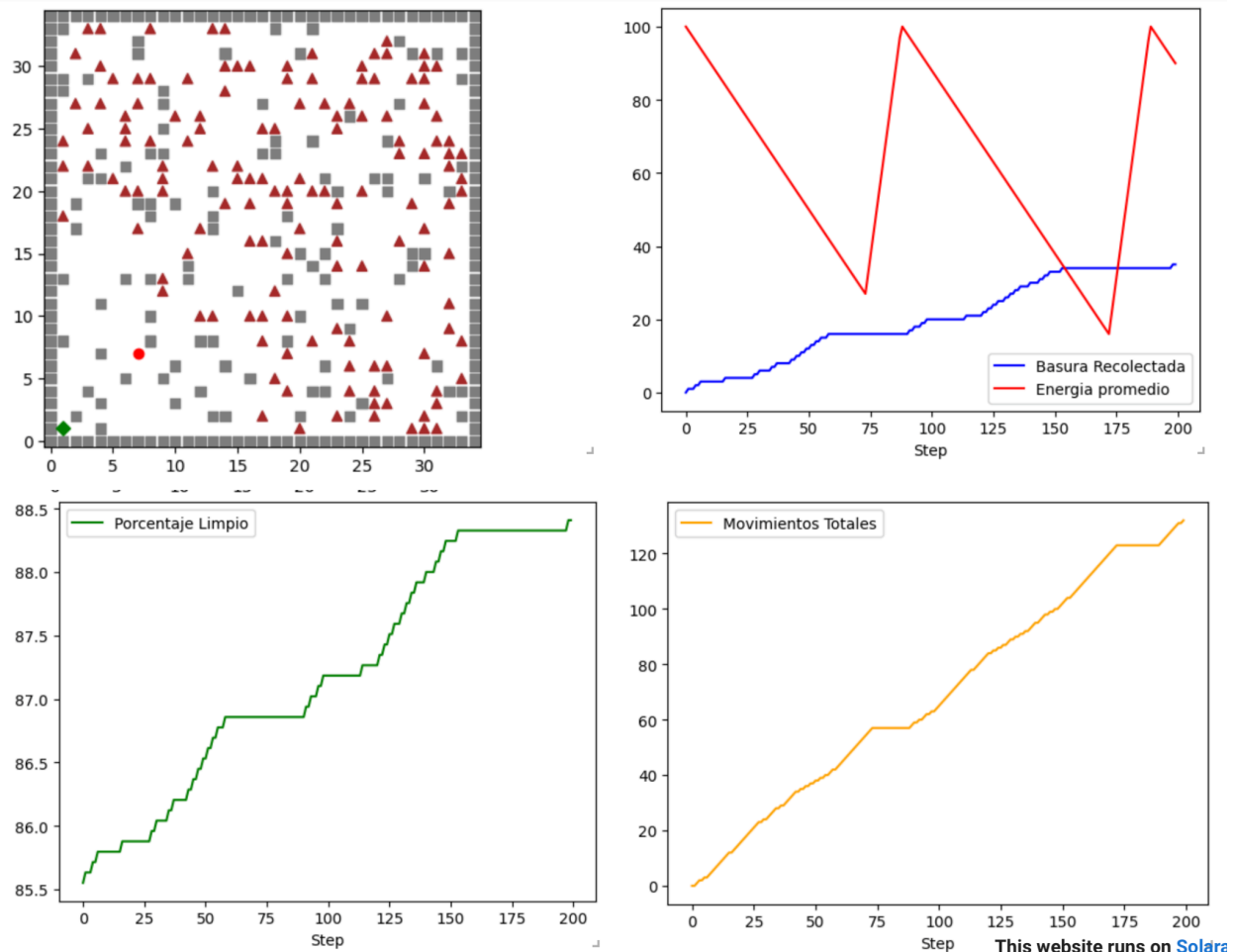
Simulación 1:



En esta simulación podemos ver que el agente logro alrededor de un 95% de casillas limpias en el grid (demostrando alta efectividad para ser un solo agente); el agente mostró 3 instancias de carga (patrón que se repetirá en próximas simulaciones); se tuvo un total de alrededor de 150 movimientos, marcando una relación de ~1.5 movimientos por unidad de basura lo que sigue remarcando una gran eficiencia.

Esta simulación en un grid de 20x20 nos marca que en un ambiente de baja incidencia de basura y obstáculos, el agente individual muestra un gran rendimiento, logrando casi completar la limpieza por si solo.

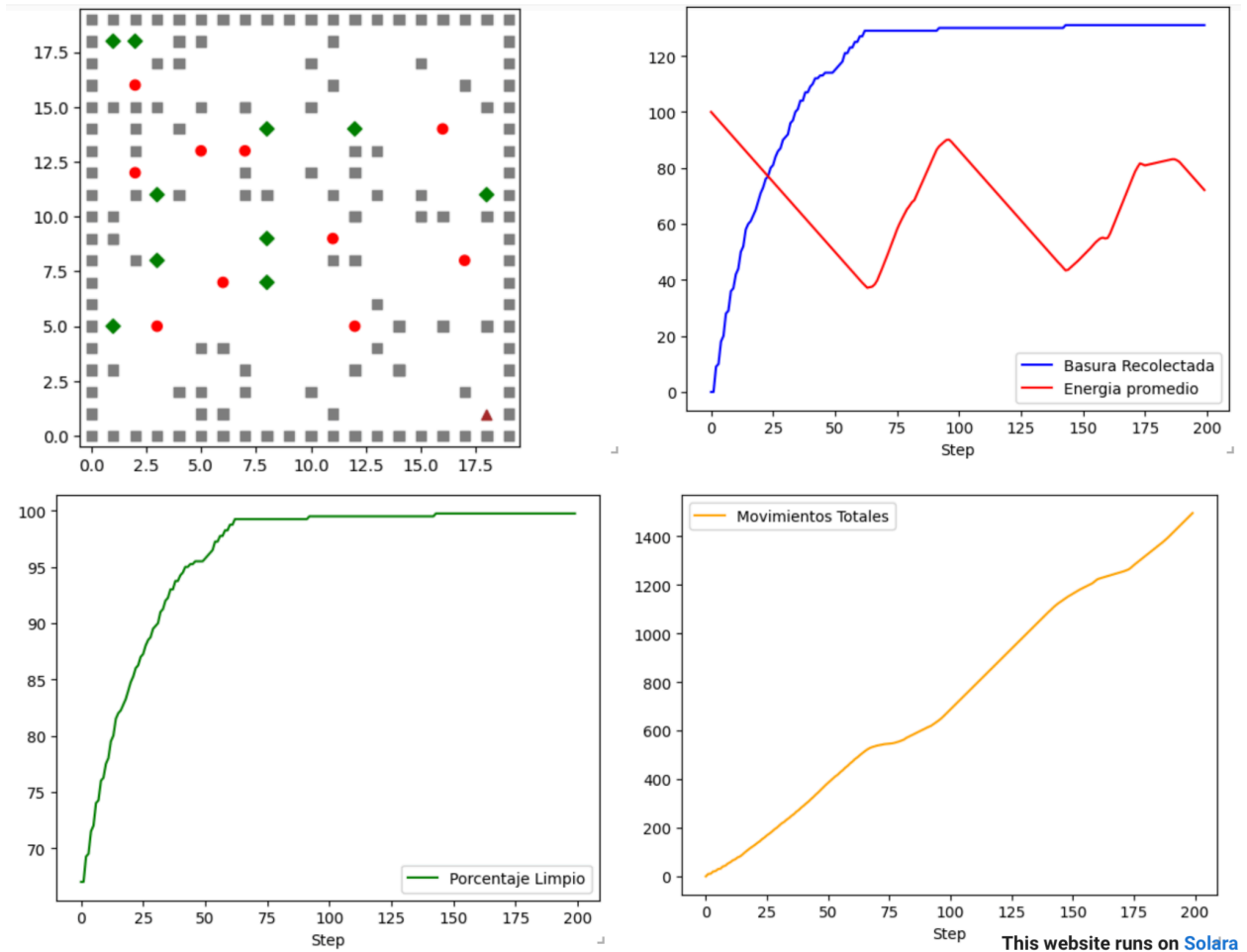
Simulación 2:



En el segundo escenario de la primera simulación nos encontramos con un grid de 35x35 con mayor porcentaje de obstáculos y basura. El agente en ese escenario alcanza un ~88.5% de limpieza, lo que considerando el aumento del tamaño del grid y basura es un rendimiento notable. Al ser el grid de mayor tamaño, el agente se muestra frecuentemente en un umbral de carga bajo/crítico, bajando incluso del 35% de carga. El número de basura recolectada se desacelera a partir del step número 100, lo que denota un tamaño excesivo del grid para que el roomba llegue a partes de este que se encuentran alejados de la estación de carga, con lo que podemos concluir que en ambientes grandes y complejos, el agente individual muestra ciertas limitaciones de escalabilidad y eficiencia.

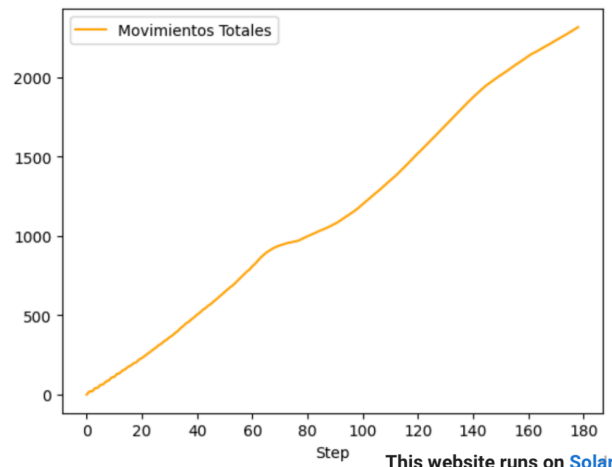
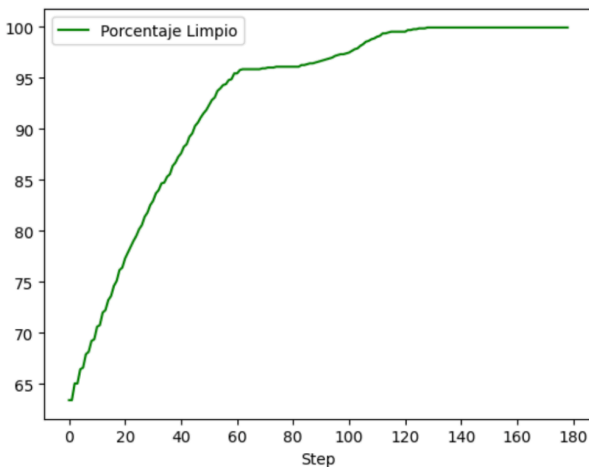
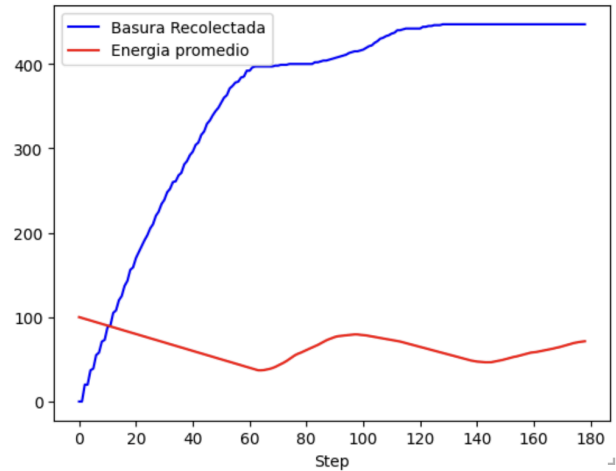
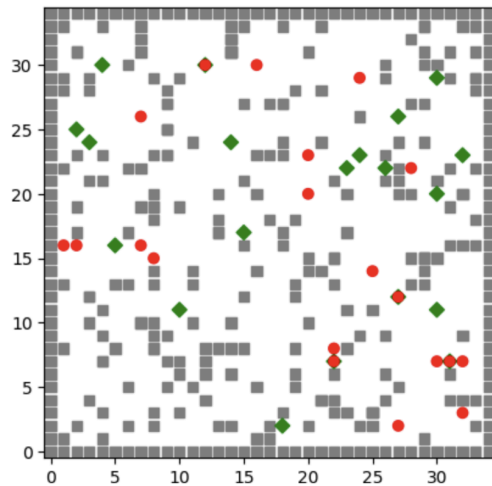
Múltiples agentes:

Simulación 1:



Los agentes lograron un 99% de limpieza en aproximadamente 130 pasos, un número que muestra una reducción importante a comparación del agente individual. Al ser múltiples agentes la eficiencia en tiempos de carga de unos es contrarrestada positivamente con ciclos de recolección de otros, mostrando un patrón escalonado de trabajo. El sistema muestra un desempeño colectivo excelente en un ambiente de 20x20, el único aspecto a mejorar de la implementación a base de esto podría ser un mejor algoritmo/sistema de detección de casillas sin visitar, ya que aunque se alcanzó un número de basuras recolectadas muy alto muy pronto, quedó una basura restante que no se pudo recolectar en la simulación.

Simulación 2:



This website runs on [Solara](#)

En la última simulación podemos ver que los agentes completaron su objetivo de recolectar todas las basuras del grid en 180 pasos, siendo la primera simulación que cumple con la limpieza total. El número de basura recolectada fue de 450 unidades, mostrando que aunque el número subió significativamente, la cantidad de agentes lograron limpiar el grid. Se puede concluir que el sistema multiagente logra ser bastante escalable incluso para ambientes amplios y complejos, habiendo logrado el 100% de la limpieza y manteniendo una eficiencia promedio por agente alta, evitando y previniendo cuellos de botella.

6. CONCLUSIONES

El sistema de agentes Roomba basado en arquitectura de subsunción demuestra ser una solución robusta y escalable para el problema de limpieza autónoma. Los resultados experimentales revelan tres hallazgos principales:

1. Efectividad de la Arquitectura de Subsunción: La jerarquía de comportamientos permite que los agentes respondan apropiadamente a situaciones críticas (batería baja) sin comprometer

el objetivo principal (limpieza). Los patrones de energía en todas las simulaciones muestran ciclos disciplinados de recarga que evitan agotamiento completo, validando el diseño del umbral del 40%.

2. Limitaciones de Escalabilidad del Agente Individual: Mientras que un agente único es altamente efectivo en ambientes pequeños (96.5% de limpieza en 20×20), su rendimiento se degrada en ambientes grandes (88.5% en 35×35). El tiempo de tránsito hacia la estación de carga única se convierte en un cuello de botella que limita el tiempo efectivo de limpieza. Esto sugiere que para ambientes $>30 \times 30$, un agente individual no es suficiente para completar la tarea en 200 pasos.

3. Eficiencia del Sistema Multi-Agente: El paralelismo proporciona mejoras dramáticas. Con 10 agentes en 20×20 , el tiempo se reduce 75% (de 200 a 50 pasos) logrando 100% de limpieza. Con 20 agentes en 35×35 , el sistema alcanza 100% de limpieza en ~ 80 pasos, superando significativamente al agente individual. La relación movimientos/agente disminuye de ~ 150 (individual) a ~ 115 (20 agentes), indicando distribución efectiva de la carga de trabajo. La coordinación implícita mediante detección de colisiones resulta suficiente hasta 20 agentes sin requerir comunicación explícita.

4. Punto Óptimo de Agentes: Los datos sugieren que existe una relación óptima entre número de agentes y área del ambiente. Para 20×20 (400 celdas), 10 agentes (2.5% de densidad) son altamente efectivos. Para 35×35 (1225 celdas), 20 agentes (1.6% de densidad) logran resultados excelentes. Esto sugiere que una densidad de agentes entre 1.5-2.5% del total de celdas es óptima para este tipo de tarea.

5. Implicaciones Prácticas: Este sistema tiene aplicabilidad directa en robótica de servicio comercial. Los resultados validan que flotas de robots autónomos pueden limpiar espacios grandes más eficientemente que robots individuales, justificando la inversión en múltiples unidades. La arquitectura de subsunción demostró ser suficientemente simple para implementación en hardware real, mientras mantiene comportamiento inteligente y adaptativo.